Trabalho do Grau B - TGB

Analisador sintático, verificação semântica e tradução dirigida

Trabalho	Analisador sintático completo, integrado ao analisador léxico, com verificação semântica mínima e tradução dirigida por sintaxe, para uma linguagem projetada e batizada pelo próprio grupo. Corresponde a 50% da nota do Grau B; os demais 50% provêm da prova do grau.
Grupo	De 1 (uma) a 4 (quatro) pessoas.
Ferramenta	Flex e Bison, PLY (Python Lex-Yacc) ou Lark, à escolha do grupo, com os mesmos critérios de avaliação. Todas são gratuitas e executam no Google Colab. Os conjuntos estão detalhados na seção 10.
Entrega	No Moodle, na tarefa correspondente. A data e o horário-limite constam da própria tarefa.
Apresentação	Presencial, no dia da entrega, de 8 a 12 minutos por grupo, com exibição de vídeo do analisador em execução.

1. Objetivo

Projetar uma pequena linguagem de programação e construir um tradutor funcional para ela: analisador sintático completo integrado ao analisador léxico, verificação semântica mínima e uma tradução dirigida por sintaxe demonstrável. O trabalho fecha o percurso do front-end de um compilador, do texto do programa até uma representação intermediária que outro estágio poderia consumir.

2. A linguagem a ser reconhecida

Cada grupo projeta a sua própria linguagem e lhe dá um nome. Este enunciado fixa as construções que ela precisa oferecer, mas não as palavras reservadas nem os símbolos: onde se lê comando condicional, o grupo pode adotar `if`, `If`, `when` ou `When`; onde se lê tipo inteiro, pode adotar `int`, `Int`, `integer` ou `Integer`. A exigência é que as escolhas sejam coerentes entre si e estejam documentadas no relatório.

As construções obrigatórias são as seguintes:

a) Declaração de variáveis, com ao menos dois tipos distintos, um numérico inteiro e outro numérico fracionário, em declarações individuais ou em lista.

b) Comando condicional, com bloco alternativo opcional.

c) Comando de seleção múltipla, com casos unitários ou múltiplos, encerramento de caso e caso padrão opcional.

d) Expressões aritméticas de soma, subtração, multiplicação e divisão, com precedência e associatividade corretas.

e) Atribuição a variáveis declaradas, encerrada pelo delimitador de comando que o grupo definir.

f) Condições com os seis operadores relacionais: igual, diferente, maior, menor, maior ou igual e menor ou igual.

Nome da linguagem

O grupo batiza a linguagem. O nome identifica o trabalho e aparece no relatório, na apresentação e nas mensagens do analisador.

Escolha das palavras reservadas

As palavras reservadas são usualmente escritas em inglês, como em `if`, `else`, `while`, `switch`, `case`, `break`, `return`, `int`, `float` e `void`. A escolha é do grupo, desde que o critério seja único e esteja documentado no relatório.

A grafia também é decisão do grupo: `int`, `Int`, `integer`, `Integer` e `INTEGER` são escolhas igualmente legítimas, e nada impede reservar a inicial maiúscula para os nomes de tipo, no estilo das linguagens orientadas a objetos, caso o grupo pretenda estender a linguagem mais adiante. O relatório deve registrar se a linguagem distingue maiúsculas de minúsculas, porque disso depende a especificação do analisador léxico.

Exemplo de programa válido

O trecho abaixo está escrito em Beacon, uma linguagem fictícia criada apenas para ilustrar o formato. As palavras reservadas são escolha de quem a projetou: outro grupo escreveria as mesmas construções com outras palavras, e o programa continuaria válido.

Programa aceito por Beacon, uma linguagem fictícia

```
Integer limite, faixa;
Decimal media;

limite = 10 + 5 * 2;

when (limite > 20) {
    media = limite / 2;
} otherwise {
    media = 0;
}

match (faixa) {
    option 1: limite = limite + 1; stop;
    fallback: limite = 0;
}
```

3. Requisitos de implementação

a) Gramática completa documentada em arquivo editável, com o nome da linguagem, as palavras reservadas e os símbolos adotados, e as decisões de precedência e fatoração justificadas.

b) Analisador sintático executável integrado ao analisador léxico, reconhecendo os programas de teste.

c) Verificação semântica mínima: variável declarada antes do uso e compatibilidade de tipos em atribuições, com mensagens e número da linha.

d) Tradução dirigida por sintaxe demonstrável: notação pós-fixada das expressões, código de três endereços ou árvore sintática abstrata (AST) impressa.

e) Programas de teste: no mínimo quatro válidos e quatro inválidos, sintáticos e semânticos, com as saídas comentadas.

f) Relatório no template da disciplina e apresentação de 8 a 12 minutos, com exibição do vídeo da demonstração.

4. Demonstração em vídeo

A demonstração do analisador em execução deve ser apresentada por meio de vídeo previamente gravado, em formato MP4, e não ao vivo, em razão das restrições de instalação de software nos computadores da sala e do tempo reduzido da apresentação. O vídeo dispensa áudio, pois a execução é explicada pelo grupo durante a exibição, e deve ter duração de até 1 (um) minuto. A gravação pode ser feita com qualquer ferramenta de captura de tela ou de reunião que exporte em MP4.

O vídeo deve mostrar na tela, em sequência: um programa válido sendo reconhecido, a saída da tradução dirigida por sintaxe e ao menos um programa inválido de cada natureza, sintática e semântica, com a mensagem

de erro e o número da linha. Quando o analisador executa no Google Colab, basta gravar a tela do próprio caderno.

5. Entregáveis

A entrega é feita pelo Moodle, na tarefa correspondente, dentro do prazo ali indicado, e deve conter:

1. Gramática documentada, completa, com o nome da linguagem, as palavras reservadas escolhidas, a precedência e os conflitos justificados.
2. Analisador executável: arquivos `.l` e `.y` ou script em Python, em execução no Google Colab sem ajustes.
3. Programas de teste: quatro válidos e quatro inválidos, com as saídas comentadas.
4. Vídeo da demonstração, em formato MP4.
5. Relatório no template da disciplina, disponível no mesmo tópico do Moodle.

6. Critérios de avaliação

Critério	O que se espera	Peso
Gramática e integração entre os analisadores	Linguagem documentada e gramática completa, sem conflitos não justificados, em execução no Google Colab.	3,0
Verificação semântica	Erros detectados com mensagens claras e número da linha.	2,0
Tradução dirigida	Saída correta e explicada no relatório.	2,0
Testes e análise	Cobertura dos casos e discussão dos resultados.	2,0
Apresentação	Vídeo claro, explicação segura e perguntas respondidas com domínio.	1,0

O relatório documenta o que foi construído e não substitui a implementação.

7. Apresentação

A apresentação ocorre presencialmente no dia da entrega, dura de 8 a 12 minutos por grupo e organiza-se em três atos: primeiro, a exibição do vídeo, com o analisador em execução; depois, as decisões de projeto, da linguagem escolhida à gramática construída; por fim, a tradução produzida e o tratamento dos casos de erro. Seguem-se as perguntas da turma e do professor.

8. Observações sobre conflitos

Conflitos shift/reduce fazem parte do processo de construção de um analisador ascendente e não invalidam o trabalho. O gerador informa quantos existem e produz o analisador mesmo assim, resolvendo-os por uma regra padrão. O aviso, porém, nunca deve ser ignorado: cada conflito encontrado precisa ser documentado no relatório, com a explicação de como foi tratado. Conflitos reduce/reduce indicam gramática mal projetada e devem ser corrigidos.

9. Regras e observações

- Uso de inteligência artificial (IA): permitido como assistente, com declaração no relatório. Cada linha entregue precisa ser explicada na apresentação.
- Erros: toda mensagem de erro, sintática ou semântica, deve informar o número da linha em que ocorreu.
- Entrega: no Moodle, dentro do prazo indicado na tarefa. Envios posteriores são registrados como atraso.

10. Materiais de apoio

Sugerem-se os três conjuntos de ferramentas a seguir, cabendo a cada grupo adotar um deles por inteiro. Todos são gratuitos e levam ao mesmo resultado, e a escolha não altera os critérios de avaliação.

Conjunto	O que cada ferramenta faz	Onde executa
Flex e Bison	Flex gera o analisador léxico e Bison, o sintático. O código produzido é em C, e as ações semânticas são escritas na própria linguagem.	Google Colab ou máquina local
PLY	Um único pacote em Python: o módulo lex gera o analisador léxico e o módulo yacc, o sintático, com a mesma divisão de Flex e Bison.	Google Colab, sem instalação
Lark	Biblioteca em Python na qual uma única gramática descreve o léxico e o sintático, sem separar os dois arquivos.	Google Colab, sem instalação

Documentação oficial de cada ferramenta:

- Flex: `westes.github.io/flex/manual`
- Bison: `gnu.org/software/bison/manual`
- PLY: `ply.readthedocs.io`
- Lark: `lark-parser.readthedocs.io`
- Google Colab: `colab.research.google.com`

Avisos, orientações complementares e entregas ficam no Moodle da disciplina.